

BCC1313
ALGORITHMS AND COMPLEXITY

Hashing & Lookup

INTRODUCTION

- **Hashing** is a technique in data structures used to map data of arbitrary size to fixed-size values, usually for fast data retrieval.

- It’s widely used in applications where quick access to data is crucial, like in databases, caches, and lookup tables.

HASHING

- A hash function is the core of hashing—it converts keys (such as strings or numbers) into hash values, which are typically indices of an array or table, enabling efficient storage and retrieval.

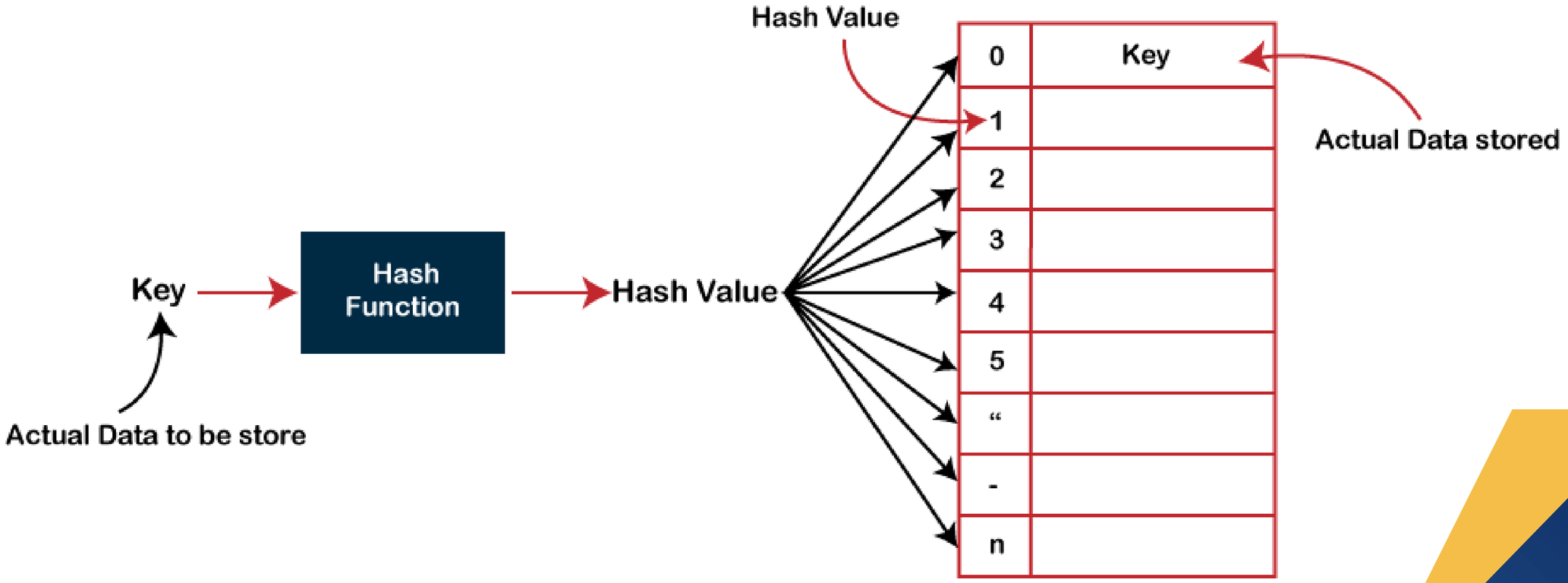
KEY CONCEPTS

- **Hash Function**
  - A function that takes input (key) and produces a fixed-size integer, often used as an index for storing the associated value in a hash table.
  - A good hash function distributes keys uniformly to avoid clustering and reduce the chances of collision.

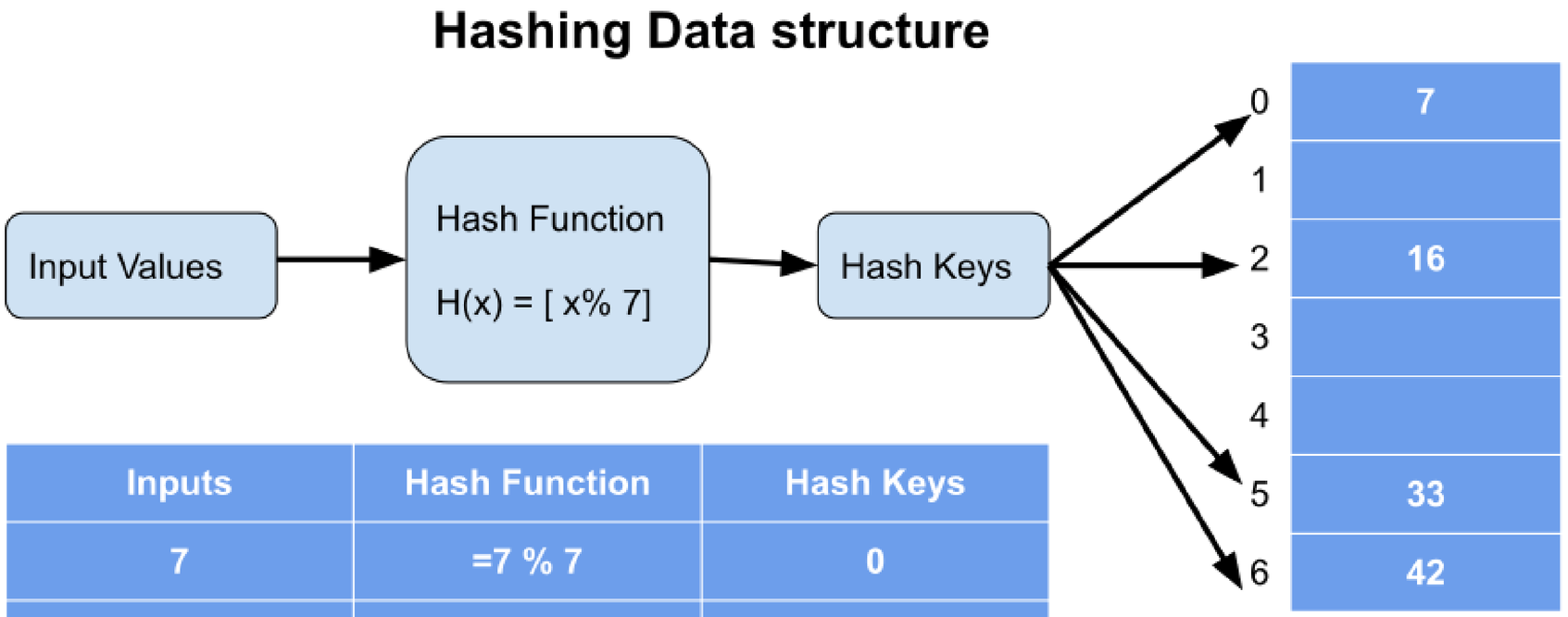
KEY CONCEPTS

- **Hash Table:** A data structure that stores data in an associative manner using a key-value pair. The key is hashed to get an index, and the value is stored at that index.
- **Collision:** When two keys are hashed to the same index, it’s known as a collision. Collisions are inevitable in hash tables due to limited array size and the vast number of potential keys.

KEY CONCEPTS



EXAMPLE



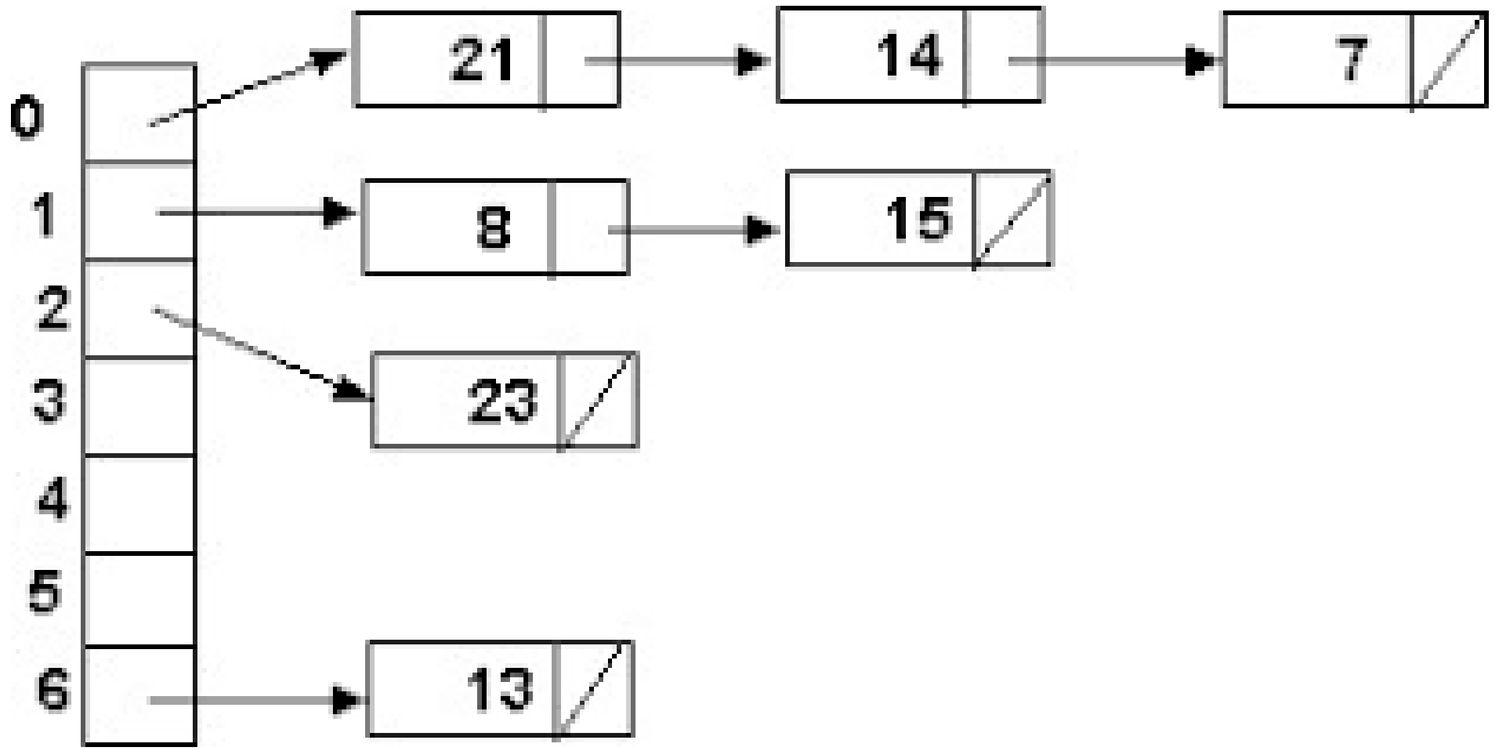
Inputs	Hash Function	Hash Keys
7	=7 % 7	0
16	=16 % 7	2
33	=33 % 7	5
42	=42 % 7	6

HANDLING COLLISIONS

1. Chaining (Separate Chaining):

- In this technique, each position in the hash table holds a linked list (or a tree).
- When a collision occurs (i.e., two keys map to the same index), the new value is added to the linked list at that index.
- Example: If both keys A and B hash to the same index i, instead of replacing one with the other, both A and B are stored as nodes in the linked list at index i.

HANDLING COLLISIONS



HANDLING COLLISIONS

Advantages:

- Simple to implement.
- Can handle an unlimited number of collisions at the same index.

Disadvantages:

- Requires additional memory for storing linked lists.
- Performance can degrade if many collisions occur, turning the list into a long chain.

HANDLING COLLISIONS

2. Open Addressing:

- In open addressing, if a collision occurs, the algorithm searches for the next available empty slot in the hash table to place the value.
- Types of open addressing:
  - **Linear Probing**: If a collision occurs at index i, the algorithm checks i+1, then i+2, and so on, wrapping around the table if necessary.
  - **Quadratic Probing**: After a collision at index i, the algorithm checks i+1^2, i+2^2, i+3^2, and so on.
  - **Double Hashing**: Uses a second hash function to calculate the next probe location in case of a collision.

HANDLING COLLISIONS

Advantages:

- More memory-efficient than chaining, as there are no extra data structures like linked lists.
- Faster lookups if the load factor is low.

Disadvantages:

- Can lead to clustering (e.g., long sequences of filled slots), degrading performance.
- More complex to implement than chaining.

HANDLING COLLISIONS

Example:

Values: 15, 7, 22, 37, 2 and the hash key is hash(x) = x % 5)

Steps:

- Inserting 15: 15 % 5 = 0. The value 15 is placed at index 0.
- Inserting 7: 7 % 5 = 2. The value 7 is placed at index 2.
- Inserting 22: 22 % 5 = 2. A collision occurs since index 2 already contains 7. Using chaining, we add 22 to the linked list at index 2.
- Inserting 37: 37 % 5 = 2. Another collision at index 2. Add 37 to the linked list at index 2.
- Inserting 2: 2 % 5 = 2. Another collision. Add 2 to the linked list at index 2.

HANDLING COLLISIONS

Example (with chaining):

Values: 15, 7, 22, 37, 2 and the hash key is hash(x) = x % 5)

Index	Values
0	15
1	(empty)
2	7 → 22 →37 → 2
3	(empty)
4	(empty)

Steps:

- Inserting 15: 15 % 5 = 0. The value 15 is placed at index 0.
- Inserting 7: 7 % 5 = 2. The value 7 is placed at index 2.
- Inserting 22: 22 % 5 = 2. A collision occurs since index 2 already contains 7. Using chaining, we add 22 to the linked list at index 2.
- Inserting 37: 37 % 5 = 2. Another collision at index 2. Add 37 to the linked list at index 2.
- Inserting 2: 2 % 5 = 2. Another collision. Add 2 to the linked list at index 2.

HANDLING COLLISIONS

Example (with open addressing)

Values: 15, 7, 22, 37, 2 and the hash key is hash(x) = x % 5)

Index	Values
0	15
1	2
2	7
3	22
4	37

Steps:

- Inserting 15: 15 % 5 = 0. Place 15 at index 0.
- Inserting 7: 7 % 5 = 2. Place 7 at index 2.
- Inserting 22: 22 % 5 = 2. A collision occurs at index 2. Use linear probing and check index 3. Place 22 at index 3.
- Inserting 37: 37 % 5 = 2. A collision at index 2. Use linear probing and check index 3 (occupied), then check index 4. Place 37 at index 4.
- Inserting 2: 2 % 5 = 2. A collision occurs. Check index 3 (occupied), check index 4 (occupied), and check index 0 (occupied). Wrap around to index 1 and place 2 at index 1.

APPLICATIONS OF HASHING

- Hashing is highly effective for scenarios requiring quick access and data retrieval:
  - **Databases:** Used in indexes and lookup tables to allow quick search and access.
  - **Caches:** Hash tables are used to manage caches, storing recently accessed data for fast retrieval.

APPLICATIONS OF HASHING

- **Symbol Tables in Compilers:** Hashing enables fast storage and retrieval of variable names and associated values.
- **Password Storage:** Passwords are hashed for secure storage, with the hash value stored rather than the password itself.
- **Data Deduplication:** Hashing helps detect duplicate files or records by comparing hash values rather than the actual data.

ADVANTAGES

- Advantages of Hashing includes:
  - **Fast Access:** Enables near-constant time complexity $O(1)$O(1) for insertions, deletions, and lookups, assuming a good hash function and low collision rate.
  - **Memory Efficiency:** Stores only necessary data by mapping keys to specific indices.

DISADVANTAGES

- The disadvantages are:
  - **Collisions:** Poor hash functions or high load factors increase collisions, slowing down performance.
  - **Fixed-Size Output:** Can be memory-inefficient if data doesn’t fit well within hash table bounds, requiring resizing.
  - **Not Sorted:** Hashing doesn’t maintain any order of keys, making it unsuitable for ordered data retrieval.

DISADVANTAGES

- The disadvantages are:
  - **Collisions:** Poor hash functions or high load factors increase collisions, slowing down performance.
  - **Fixed-Size Output:** Can be memory-inefficient if data doesn’t fit well within hash table bounds, requiring resizing.
  - **Not Sorted:** Hashing doesn’t maintain any order of keys, making it unsuitable for ordered data retrieval.

SUMMARY

- In summary, hashing is a powerful tool for data indexing and retrieval, provided that a good hash function and collision-handling strategy are in place.
- The process is using a hash function, which maps the input data to a specific location in a hash table.
- Hashing is widely used in various applications, including database indexing, caching, and password storage.